

18. API Description

18.1 The Contract as an Artefact

The interface between the browser and the service is defined in `docs/api-contract.json`, and that file is treated as authoritative rather than descriptive. The backend's Pydantic schemas and the frontend's TypeScript types are both required to match it, and any change to an endpoint is made to the contract first.

This discipline was adopted after a failure, not in anticipation of one. The two sides had been developed against separate mental models of the interface, and six divergences accumulated before anything was tested end to end: the frontend expected a flat prediction object where the backend returned an envelope; it read a field named `confidence` where the backend emitted `risk_confidence`; it posted assessments to `/assessments` where the route was `/assessment`; and three of the analytics field names disagreed. Each was individually trivial and collectively fatal — the application compiled, the backend passed its own tests, and no request succeeded. Consolidating the interface into one document that both sides are checked against removed the class of error rather than the instances of it.

18.2 Endpoints

Seven routes are served from `http://localhost:8000`.

Method	Path	Persists	Response
GET	<code>/health</code>	no	bare object
GET	<code>/model/info</code>	no	bare object
POST	<code>/predict</code>	no	enveloped
POST	<code>/assessment</code>	yes	enveloped, with <code>assessment_id</code>
GET	<code>/assessment/{id}</code>	no	enveloped, with <code>assessment_id</code>
GET	<code>/assessments</code>	no	enveloped list, with <code>total</code>
GET	<code>/analytics</code>	no	bare object

Interactive documentation is generated automatically by FastAPI from the Pydantic schemas and served at `/docs`. Because the schemas are the same objects that perform validation, the published documentation cannot drift from the enforced behaviour.

18.3 Response Envelope

The four prediction and assessment endpoints return:

```
{ "success": true, "data": { ... } }
```

`POST /assessment` and `GET /assessment/{id}` add `assessment_id` as a sibling of `data`, not inside it. `GET /assessments` substitutes `total` and `items` for `data`.

The remaining three endpoints return their payload unwrapped. This is an inconsistency rather than a design decision, and it should be recorded as such: a client cannot determine from the shape of a response whether it needs to unwrap. The frontend absorbs the difference inside `src/api/assessments.ts`, where `postPredict` returns `response.data.data` while `getAnalytics` returns `response.data`. No page component sees the distinction, so the cost is contained, but a second consumer of this API would have to read the contract to discover which convention applies where.

18.4 Request Schema

All four write endpoints accept the same body, twenty-six fields validated by `AssessmentInput`. Categorical fields are `Literal` types constrained to the exact strings present in the training vocabulary; numeric fields carry explicit bounds. The model declares `extra = "forbid"`, so an unrecognised key is an error rather than an ignored field.

An example request:

```
curl -X POST http://localhost:8000/predict \
-H 'Content-Type: application/json' \
-d '{
  "age": 30, "gender": "Male", "education": "Graduate",
  "occupation": "Salaried", "income_level": "Middle",
  "marital_status": "Single", "dependents": 0,
  "location": "Urban", "employment_type": "Full-Time",
  "years_of_experience": 5,
  "monthly_income": 50000, "monthly_expenses": 35000,
  "savings_rate": 30, "emergency_fund_months": 3,
  "total_debt": 100000, "loan_amount": 500000,
  "credit_score": 720, "investment_experience_years": 3,
  "investment_frequency": "Monthly", "insurance_coverage": "Basic",
  "shopping_frequency": "Monthly", "online_spending_pct": 30,
  "luxury_spending_pct": 10, "subscription_count": 3,
  "gaming_expenses_monthly": 500, "travel_expenses_annual": 20000
}'
```

Note that `savings_rate` is supplied as a percentage. The inference module converts it to the fraction the model was trained on, as Section 13.5 describes.

18.5 Response Schema

The `data` object returned by `/predict`, `/assessment` and `/assessment/{id}` has twelve keys:

Key	Type	Meaning
risk_category	string	Low, Medium or High
risk_confidence	float	probability of the predicted class, 0–1
risk_probabilities	object	probability across all three classes
investment_preference	string	FD, Mutual Funds, Stocks, Gold or Crypto
investment_confidence	float	0–1
financial_decision_score	float	0–100
behavioral_composite_score	float	0–100
financial_discipline_score	float	0–100
shap_values	object	ten features → attribution magnitude (always ≥ 0 ; see §21.4)
feature_importance	object	ten features → global importance
recommendations	array	3–5 strings, rule-generated
behavioral_biases	array	rule-detected, may be empty

`investment_preference` and `investment_confidence` are returned by the API and rendered by the interface, but the model producing them does not work. Section 22 shows that its accuracy of 0.259 falls below the 0.287 majority-class baseline, and that no model could do materially better given how the label was constructed. The fields remain in the contract; the report does not claim they are meaningful.

18.6 Error Handling

Status	Condition
422	request body fails schema validation; response names the offending field
404	GET <code>/assessment/{id}</code> for an unknown identifier
500	prediction raised during <code>POST /assessment</code> ; the assessment row is marked <code>failed</code>
503	<code>POST /predict</code> when the inference module failed to import at startup

Validation errors are the common case and are informative. Submitting `"education": "Bachelor's"` — a value the interface once offered but the model has never seen — produces a 422 naming `education` and listing the four permitted literals. This is the correct behaviour: a silently accepted synonym would be encoded as the training median and would produce a plausible-looking prediction from an input the model cannot represent.

GET `/model/info` is an exception to the table. When it cannot locate the metadata file it returns `{"error": "..."} with HTTP 200 rather than a 5xx. Because the path it consults is hardcoded to an absolute location on the development machine, this is the response it produces under Docker Compose. Section 16.7 records the defect.`

18.7 Analytics

`GET /analytics` aggregates over stored assessments and returns totals, the distribution of predicted risk categories and investment preferences, the mean of each of the three scores, and a per-day assessment count.

The per-day count is the one query that could not be written portably. SQLite truncates a timestamp with `strftime('%Y-%m-%d', ...)`; PostgreSQL requires `to_char(..., 'YYYY-MM-DD')` and rejects the former outright. The service inspects `db.bind.dialect.name` and emits the appropriate expression. Before this was introduced the endpoint returned HTTP 500 under Docker Compose while passing its tests against SQLite locally — a failure mode worth noting, since the test suite exercised a database the deployment does not use.

18.8 Cross-Origin Policy and Authentication

CORS permits `http://localhost:5173` and `http://localhost:3000`, matching the Vite development server and the nginx container respectively, with credentials enabled and all methods and headers allowed.

No authentication exists. There are no credentials, no sessions and no tokens; every endpoint is unauthenticated and every stored assessment is anonymous. The `users` table described in Section 17 is never written to for exactly this reason. The consequence is that `GET /assessments` returns every assessment any visitor has ever submitted, including the twenty-six financial fields of each, to any caller. For a demonstration system with synthetic inputs this is tolerable. For a system carrying real financial disclosures it would not be, and Section 26 records it as the most serious limitation of the implementation.

Screenshot 18.1 — *Swagger UI at `/docs`, all seven endpoints collapsed and visible in one frame. Place in Section 18.2.*

Screenshot 18.2 — *An expanded `POST /predict` in Swagger, showing the generated request schema with the `Literal` enumerations. This demonstrates that the published documentation is the enforced schema. Place in Section 18.4.*

Screenshot 18.3 — *A 422 response, produced by submitting `"education": "Bachelor's"`. Place in Section 18.6.*

Figure 18.1 — *Sequence diagram for `POST /assessment`, spanning browser, router, schema validation, inference module and database. This is Figure 11.2; render it once from the Mermaid source in Section 11.2 and cross-reference.*